



TECHNICAL WRITEUP

The system, and why it's built to be honest.

It finds the best of five landing pages, learns what actually drives conversion, and designs a sixth that beats them. The part worth your attention is the measurement: nothing is called a win until the data says so, and every number is reproducible.

Hari Rajashekar

Live app: schole-growth-lab.vercel.app

A companion to the explainer and pitch videos.

Read this if you want the design decisions and the exact numbers.

What this is, in one paragraph

The task was open-ended, so I framed it around two questions a real growth team actually has to answer. First: of the pages we already have, which one converts best, and are we sure enough to shift budget onto it? Second: what actually makes a page convert, and can we use that to build a better one? The app answers both, and it treats the answering as the hard part. A growth system that hands you a confident wrong number is worse than one that says "not yet." That single constraint drove most of the design decisions below, and it's the same bet Scholé's own product makes every time it routes a lead.

HOW TO READ THIS

Sections 1 to 3 are the setup and architecture. Sections 4 and 5 are the two engines and their results. Section 6 is the honest accounting, which is the part I'd actually defend the project on. Section 7 is how the design got there (it started rigged, and fixing that was the real work). Section 8 lists every assumption I made and where the method's limits are.

The problem, and the constraint I put first

Five landing pages sell the same product with different angles. The job is to figure out what works and improve it. The interesting decision was what to optimize for, and I chose trustworthiness of the conclusion over raw lift.

Splitting the work into two tracks fell out naturally. Track 1 is a selection problem: given the pages that exist, find the best one while wasting as little traffic as possible confirming it. Track 2 is a modeling problem: learn the effect of each design ingredient, then synthesize a new page that maximizes predicted conversion. They sound similar and they are not. One picks among things that exist; the other builds something that doesn't yet.

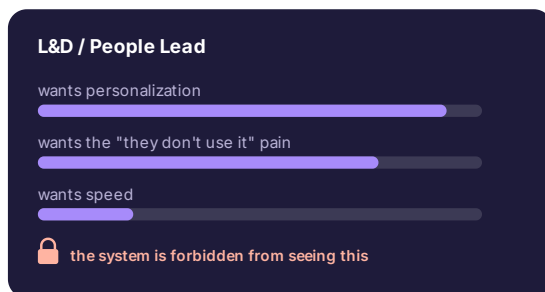
Underneath both, I made honesty a hard requirement rather than a nice-to-have. Concretely that meant three rules I held throughout: every headline number is reported against an explicit baseline, no page is declared a winner without a significance gate, and every result is seeded so it reproduces exactly. Those rules cost some flattering numbers. They're what makes the rest of the document worth reading.

A known-answer testbed

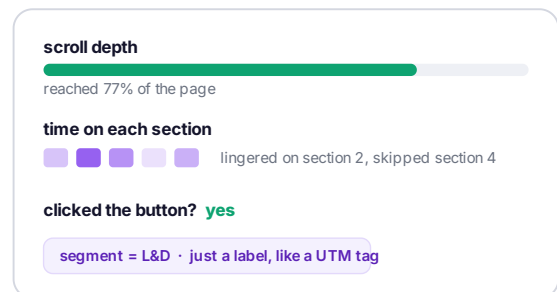
Real traffic doesn't hand you ground truth on day one, and I wanted to prove the machinery works before trusting it. So I built a simulated market where I know the answer, and then checked whether the system could recover it blind.

Three buyer personas drive the simulation, mixed 30 / 40 / 30. Each has fixed hidden preferences: a skeptical exec who wants ROI and proof, an L&D lead who wants role personalization, a hands-on IC who wants speed. Those preferences decide whether a simulated visitor clicks. The system never sees them. All it observes is behavior, scroll depth, per-section dwell, bounce, and click, plus a coarse segment label, the kind of thing a UTM tag would give you in production.

THE TRUTH: a visitor's hidden type



WHAT THE SYSTEM SEES: behavior only



BLINDFOLD

the job: rebuild the hidden preferences on the left using only the behavior on the right

The blindfold is the whole design. The visitor's true type and its preferences sit on the left, sealed off. The system only ever sees the behavioral signals on the right. The test is whether it can reconstruct the hidden preferences from behavior alone. If it can here, where I planted the answer, the same pipeline is trustworthy on real traffic where nobody knows it.

This is the one framing worth stating out loud, because it pre-empts the obvious objection ("you wrote the personas, so of course it finds them"). The point isn't that the answer exists. It's that the method recovers an answer from behavior without being shown it. Validate the detector on a coin you buried, then trust it on ground you haven't dug.

✦ ONE DELIBERATE CHOICE THAT MATTERS LATER

Simulated conversion depends only on the *structural* features of a page (angle, CTA, proof, length), never on the generated copy. That's intentional. It means the language model that writes the page text is physically incapable of inflating the measured result, which becomes the cleanest guarantee in the whole system (Section 5).

Two engines on a shared simulator

Both tracks run on the same simulated market from Section 2. Track 1 is a multi-armed bandit that finds the best existing page. Track 2 fits a response model over a designed experiment and synthesizes a new page. The map shows how they relate.

The five pages are the same product under five angles, each with its own CTA and layout:

PAGE	ANGLE	PRIMARY CTA
A	ROI & proof	Book a demo
B	Adoption-gap pain	Book a demo
C	Role personalization	Book a demo
D	Research credibility	Learn more (soft)
E	Speed / micro-learning	Try it now

THE HIDDEN WORLD

The answer key you wrote.
The system never sees it,
it only emits behavior.

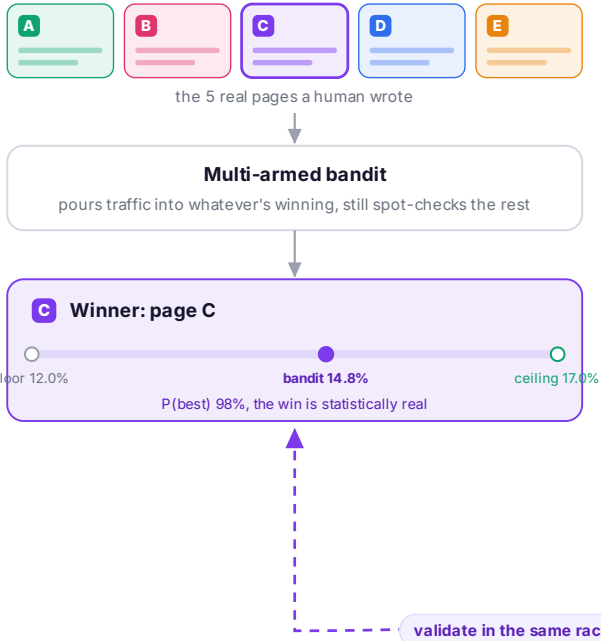
Skeptical Exec
loves ROI + proof
30% of traffic

L&D / People Lead
loves personalization
40% of traffic

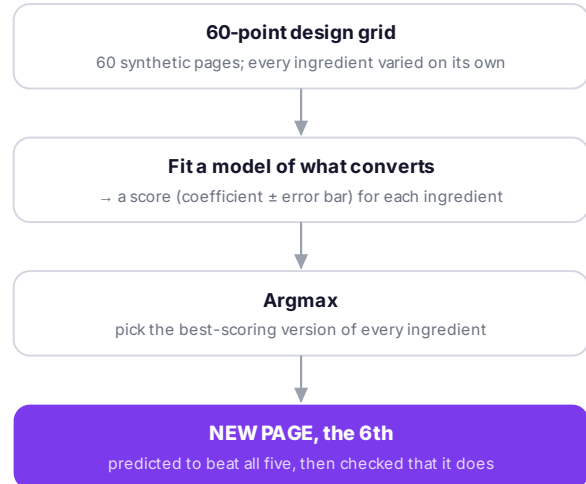
Hands-on IC
loves speed
30% of traffic

--- the blindfold: only behavior (clicks · scroll · dwell) + a segment label crosses this line, and it feeds both tracks ---

TRACK 1: pick the best of the 5 you already have



TRACK 2: learn the recipe, build a new page



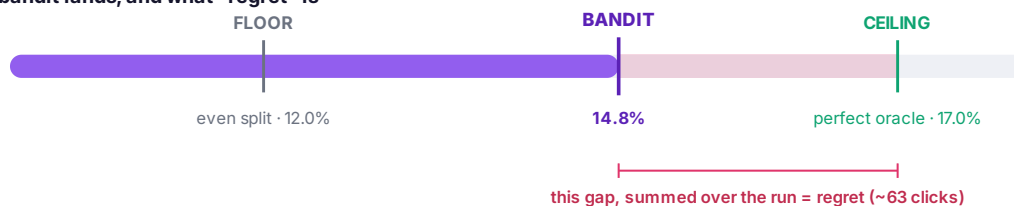
The system end to end. The persona-driven simulator produces behavior; the bandit consumes it to rank the five real pages; a separate factorial experiment feeds a fitted model that proposes and validates a new page. Everything downstream of the blindfold sees behavior only.

The bandit, framed by honest baselines

Finding the best of five pages is a bandit problem, not an even A/B split. The reason is waste: an even split keeps pouring traffic onto pages that are visibly losing for the entire test. Thompson sampling reallocates toward the leader as evidence accrues while still exploring, so you buy your answer with fewer wasted impressions.

Configuration: Thompson sampling over the five pages, 14 rounds of 300 visitors (4,200 total), seed 42 for reproducibility. Blended conversion across the run is **14.81%**. On its own that number means nothing, so I bracket it.

Where the bandit lands, and what "regret" is



Floor, bandit, ceiling. The floor (12.03%) is a naive even split that never learns. The ceiling (17.00%) is an oracle that serves the best page from the first visitor. The bandit sits between them: better than the floor because it learned, short of the ceiling because it spent some traffic learning. That shaded gap, summed over the run, is cumulative regret, about 63 clicks. Reporting the bandit rate without these two anchors would be meaningless.

Selecting a winner then requires more than the top number. I compute **P(best)** by Monte-Carlo over each page's Beta posterior (sample every page's rate repeatedly, count how often each is the max) and hold Wilson confidence intervals on each rate. A page is only called the winner once it clears 95% P(best). Page C does, at 98%. Everything else is statistically indistinguishable from the pack.

PAGE	RATE	P(BEST)	VERDICT
C Role personalization	17.0%	98%	winner · significant
B Adoption-gap pain	13.0%	1%	indistinguishable
E Speed	12.8%	1%	indistinguishable
A ROI & proof	12.7%	0%	indistinguishable
D Research	4.7%	0%	clear loser

Equal-traffic showcase pass, 1,500 visits per page, so every page gets a clean read for the leaderboard. The bandit itself runs on lopsided traffic by design; the showcase exists to report unbiased per-page rates.

Learning the recipe, then building a better page

Track 1 found the best existing page. Track 2 asks what makes any page good, and here the five real pages are useless: their ingredients are confounded, so you can't attribute the outcome to any single factor.

The soft CTA appears on exactly one page (D), which is also the research angle, which is also the worst performer. When D loses, was it the button, the angle, or the page? Unrecoverable from these five. This is the same reason you can't read ingredient effects off a handful of real landing pages in production, they co-vary.

TANGLED · your 5 real pages

each page locks its ingredients together

A	ROI	hard CTA
B	Pain	hard CTA
C	Personalization	hard CTA
D	Research	SOFT CTA
E	Speed	try-it CTA

The soft CTA shows up on exactly one page, which is also research, which is also the worst.

So when D loses: bad button? bad angle? bad page?
Impossible to say. The effects are glued together.

UNTANGLED · the 60-page grid

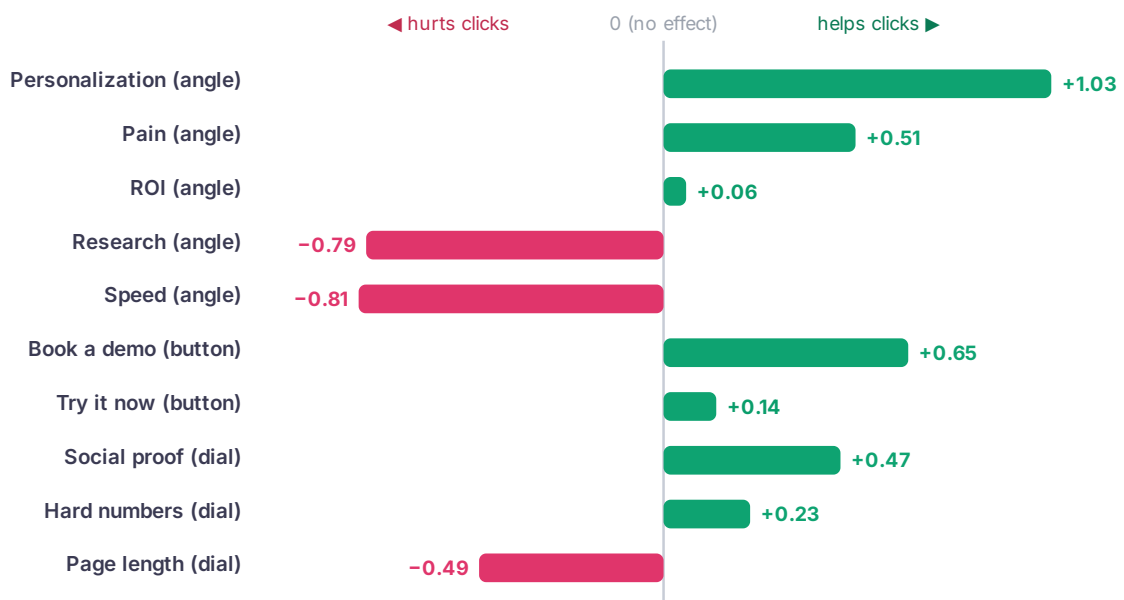
every angle is paired with every button

angle ↓ / button →	hard	soft	try-it
ROI			
Pain			
Personalization			
Research			
Speed			

15 angle×button combos, each in 4 length/proof mixes = **60 pages**. Now each ingredient's effect can be measured on its own. No humans see these.

Why the five real pages can't teach a model. On the left, each page fuses its ingredients, so effects are entangled. On the right, a 60-page factorial design crosses every angle with every CTA and every length, so each ingredient varies independently and its effect becomes identifiable. These 60 pages are internal to the experiment; no human ever sees them.

The grid is 5 angles × 3 CTAs × 4 proof/length combinations = 60 pages, run at 600 visits each (36,000 exploration visits). I fit a logistic regression to that, which gives a clean coefficient per ingredient, each with a confidence interval because the design de-confounded them.



The fitted effects, in log-odds. Role personalization is the dominant positive lever. Research and speed cost conversion on a blended audience. Book-a-demo beats the soft CTA; proof and hard numbers help; length hurts. Every coefficient carries a CI, and every one feeds the construction step below.

Building the new page is then an argmax over the fitted model: take the best-scoring setting of each ingredient and compose them. Nothing is chosen by taste.

Pick the best-scoring option in each group...

LEAD ANGLE

Personalization ✓

Pain

ROI · Research · Speed

BUTTON

Book a demo ✓

Try it now

Soft / learn more

DIALS

proof: high ✓

hard numbers: high ✓

length: short ✓

...and snap them into a new page

NEW PAGE (the 6th)

headline: personalization

support: the "they don't use it" pain

• heavy proof • real numbers

• short (5 sections)

Book a demo

The variant is assembled, not guessed. Personalization lead, pain support, book-a-demo, proof and numbers maxed, kept short. Each choice is the top-scoring option for that ingredient under the fitted model.

GENERATED VARIANT (V1)	RATE
Model's prediction	21.3%
Observed when run	19.9%
Best original in the same pass (C)	17.7%
Lift over best original	+12.4%

C reads 17.0% on the leaderboard and 17.7% here because these are two independent measurement passes with different sample draws. That's ordinary sampling variation, reported rather than smoothed over.

✦ THE LANGUAGE MODEL'S ROLE, AND WHY IT CAN'T CHEAT

An LLM writes the actual page copy, but only into a structure the coefficients already fixed. It has no say over angle, CTA, or dials. Because simulated conversion depends solely on structural features (Section 2), the copy cannot move the measured result. Concretely: with no API key the app falls back to cached copy and the numbers are identical. The model is a copywriter handed a spec, not the strategist, and that separation is what keeps the result attributable to the method rather than to prose.

6

THE PART I'D DEFEND IT ON

Honest accounting

A single run can be luck. Two checks turn "it won once" into a claim worth trusting, and both are reported with their downside visible.

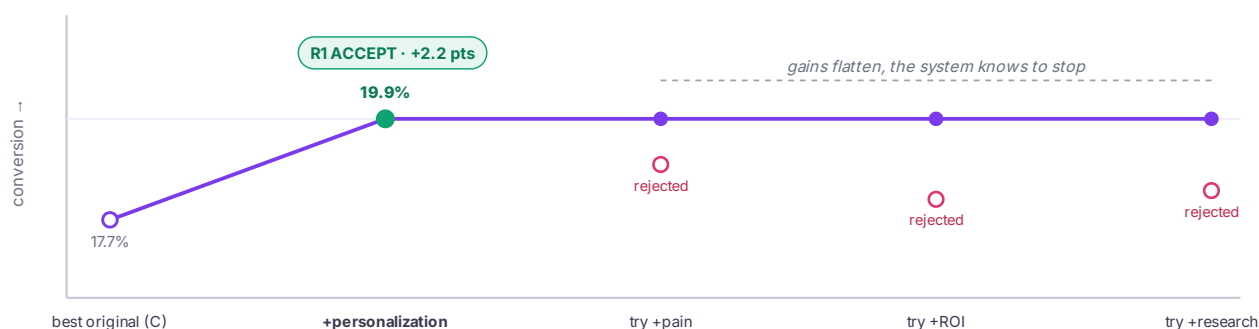
Check 1, many seeds

ACROSS 40 SEEDS	VALUE
Mean lift over best original	+18%
95% confidence interval	[-4% , +51%]
Win-rate (variant beats best original)	93%

The interval crosses zero on purpose. It says the variant is usually a clear win and occasionally a small loss, which is the truthful statement. A 93% win-rate is a strong, credible result; a system reporting 100% would be evidence the test was rigged, not evidence it was good.

Check 2, the improvement loop

The system keeps proposing the next-best idea and only keeps it if it beats the current champion. It accepts the personalization lead (17.7% → 19.9%), then rejects pain, ROI, and research emphasis in turn, and plateaus.



The plateau is the evidence. Real selection pressure means most proposals fail. The line rises once, then flattens as further changes stop beating the champion. A loop that kept climbing indefinitely would be fitting noise, not improving.

The bonus, per-segment pages

Fitting a model per segment and building a page for each does better than one page for everyone, but only where it actually helps. The exec segment converts around 42% on a tailored ROI page versus about 6% on the generic winner; the hands-on segment around 31%

on a speed page versus about 7%. The L&D segment is already best served by the generic personalization page, so there's nothing to tailor. The rule that keeps this honest: ship a targeted page only where it beats the generic one.

The first version was rigged. Fixing it was the work.

I'm including this because it's the clearest evidence of the judgment the whole project is about. The first version looked great and proved nothing, and the useful part was catching that.

Version 1 asserted its win by construction. The variant's ingredients were hard-coded to the settings I already believed were best, and those settings happened to match the personas' hidden preferences, the same preferences that decide clicks in the simulator. So the page was built from the answer key and then tested against the answer key. It was always going to win. On top of that, the "win" came from a single run, against a weak baseline, with no significance test. Reviewed honestly, it demonstrated nothing.

Version 2 removes every shortcut. The levers are learned from the de-confounded factorial fit rather than typed in, the variant is compared against the best original page, the win is gated on $P(\text{best})$, and the whole thing is validated across 40 seeds. That's the difference between a demo that flatters itself and a measurement you can act on.

V1 (ASSERTED THE WIN)	V2 (EARNED IT)
Variant levers hard-coded to the "known" best	Levers learned from a fitted model, then argmax'd
Built from the same preferences that drive clicks	Preferences rediscovered from behavior, blind
Single run, weak baseline	Compared to best original, framed by floor/ceiling
No significance	$P(\text{best}) \geq 95\%$ with confidence intervals
One seed	40-seed sweep; win-rate and interval reported

Assumptions and limitations

The task said to make assumptions where things were open and justify them. Here they are, with the reasoning, followed by where the method stops.

Assumptions I made

- **Simulated buyers stand in for real traffic.** The persona preferences and the 30/40/30 mix are illustrative. I chose values that make the problem non-trivial (no single page dominates for free) but still learnable, so the machinery has something real to recover.
- **Conversion is a function of structural features plus noise.** Copy is held causally inert deliberately, so the LLM can't inflate results and every measured effect is attributable to a design lever.
- **Calibration constants were tuned for a fair problem.** Angle weight, behavior noise, base dwell, and the scroll gate were set so the task is neither trivial nor impossible. These are levers, and the app exposes them.
- **Behavior is the only observable.** Scroll, dwell, bounce, and click plus a segment label are all the system sees, mirroring what production analytics actually provide.
- **"Win" is an in-simulation result.** It validates the method, it is not a market claim about Scholé's real pages.

Where it stops

The simulator is my model of buyers, so it's only as right as those assumptions. It validates the pipeline, not the market. With real traffic the structure is the same: the personas become real visitors, the behavior signals become real analytics, and the pipeline (bandit to find the best live page, factorial exploration to learn ingredient effects, argmax to propose, significance to gate) runs unchanged. The one thing that changes is the cost of exploration. The 60-page grid is free in a simulation; against paid traffic you'd budget it carefully, and the bandit is exactly the tool for bounding that cost. The honesty scaffolding matters more when there's real money on the line, not less.

Rerun it and you get the same numbers

Every figure in this document comes from seeded runs. Same seed, same result, every time. That's the difference between a result and a screenshot.

The logic lives in `lib/` as pure functions: the simulator and personas, the bandit with its floor/ceiling/regret and $P(\text{best})$, the factorial design, the logistic fit, and the argmax variant builder. A single conductor (`lib/lab.ts`) runs the field evaluation, the variant evaluation, the evolution loop, and the multi-seed sweep, and holds the run constants (14 rounds, 300 visitors per round, 600 exploration visits per grid page, 1,500 showcase visits, 40 seeds, bandit seed 42). Anyone can open the app, rerun the experiment, and land on the numbers reported here.

THE ONE LINE, IF YOU ONLY REMEMBER ONE

It's a landing-page system that finds your best page, learns why it works, and builds a better one, and it refuses to call any of it a win until the data actually says so. That last part is the point.